TSI Site Services — Field Tool Build Brief

From: Joe Brassell | **To:** Steve (backend), IT Guy (infrastructure) | **Date:** March 10, 2026

Overview

We're replacing the Site Services section of the current portal with a new field-optimized web app for the techs. The current portal works but the techs won't use it — it's too many pages, too many clicks. The new version puts everything on one screen, auto-saves as they type, and works well on the laptops they already have.

A fully working prototype already exists with mock data. The frontend is done. We need two things to make it real:

1. **Steve** — wire the real SQL Server database and Azure AD auth behind the existing API
2. **IT Guy** — host it so it's reachable from any hospital WiFi and set up the Azure AD app registration

Data writes directly to the same SQL Server WinScope uses. No sync, no import/export — when a tech saves a visit in the field, it's already in WinScope.

How Techs Log In

Techs open the app URL in their browser. Because they're already signed into their TSI Microsoft account on the laptop (same account as Outlook and Teams), the app authenticates them automatically — no separate password, no new account. If for some reason they're not signed in, they get the standard Microsoft login page and use their existing TSI credentials.

There are no new accounts to create or manage. Access is controlled through M365 — disable someone's TSI account and they're locked out of the field app automatically.

Architecture

```
Tech's Laptop (browser on hospital WiFi)
        │
        │  HTTPS
        ▼
   [TSI On-Prem Server]  — same server as existing portal
        │  Node/Express API  (port 443, public-facing)
        │
        ▼
   SQL Server  ◄─── WinScope reads from here too
```

Azure AD sits in front of the API — every request requires a valid TSI Microsoft token. SQL Server never touches the internet directly.

STEVE — Backend / API

Files

File	Description
`server.js`	Node/Express REST API — currently runs on in-memory mock data. Replace each data section with real SQL queries.
`public/index.html`	Complete single-page frontend. No build step. Calls the API on the same origin. Don't touch this — just make the API work.

Everything in `server.js` is clearly commented. Each mock data block maps directly to a DB table. The mock implementation is the spec — keep the same endpoint paths and response shapes and the frontend works without changes.

Database Tables

Column names were extracted directly from the live portal's ASP.NET control IDs and hidden fields. These should match your actual DB column names.

SiteServices

Column	Displays As	Notes
SiteServiceKey	—	PK
DepartmentKey	"Customer #"	**Note: what the UI shows as "Customer #" is the DepartmentKey**
TechKey	Mobile Tech	FK to Technicians
OnsiteDate	Start Dt	date
DateSubmitted	Submitted	date, null = draft
PO	PO #	
WorkOrderNumber	Work Order #	e.g. NV26006001
PricingList	Price Class	
TruckNumber	Truck Number	
SalesRep	Sales Rep	
StartTime	Start Tm	varchar HH:MM AM/PM
EndTime	End Tm	varchar
EndDate	End Dt	date

Notes	Notes / Next Steps	varchar(max)
TrayCount	Total Trays	keep in sync server-side
CalcedTotal	Calced Cost	SUM(Quantity × UnitPrice) across all trays — compute on read, never stored by frontend
TotalCost	Total Cost	separate from CalcedTotal — see questions
Cap	Cap	CapitatedCost label in live portal

Your GET queries should JOIN to Clients, Departments, and Technicians to include `ClientName`, `DepartmentName`, `TechnicianName`, `Address`, `CityStateZip`.

Status (draft/submitted) can be derived: `DateSubmitted IS NULL` = draft.

SiteServiceTrays

Column	Notes
SiteServiceTrayKey	PK
SiteServiceKey	FK
TrayNumber	Display sequence (1, 2, 3...)
TrayName	
AdHoc	**NEW column** — bit, default 0. Repair Bins and ad-hoc trays are flagged here and excluded from template sync on submit.

SiteServiceTrayItems

> ⚠️ ***Important:*** *What the UI labels "Repaired" is stored as* ***`Quantity`*** *in the DB — confirmed from ASP.NET control name* `txtQuantity`*. Alias it as* `RepairedCount` *in your SELECT so the frontend gets what it expects.*

Column	Displays As	Notes
ItemKey	—	PK
SiteServiceTrayKey	—	FK
InstrumentType	Item Description	matches Code in InstrumentTypes
UnitPrice	—	price at time of entry (snapshot)
Total	Total Instruments	`txtTotal`
Quantity	Repaired	`txtQuantity` — alias as `RepairedCount`
SentToTSI	Sent To TSI	`txtSentToTSI`
BER	Beyond Economical Repair	`txtBER`
Replaced	Replaced Count	`txtReplaced`

Inspected is never stored — frontend calculates: `Total - Quantity - SentToTSI - BER`

Row cost is never stored — frontend calculates: `Quantity × UnitPrice`

InstrumentTypes

Column	Notes
Code	instrument name — varchar PK
Price	current unit price — decimal

~90 rows. The full list with current prices is in `server.js` under `instrumentTypes`. Use it to seed if the table is missing or named differently.

TrayTemplates *(existing)*

Column	Notes
TemplateKey	PK
DepartmentKey	FK — templates are scoped per department
TrayName	
Status	'Active' or 'Inactive'

TrayTemplateItems *(new table)*

The current portal stores tray templates as names only. The new app adds per-item default counts so techs don't have to re-enter the same numbers every visit.

Column	Notes
ItemKey	PK
TemplateKey	FK
InstrumentType	
UnitPrice	
DefaultCount	pre-fills `Total` when tech loads template

When a tech picks a saved tray, items from this table are copied into SiteServiceTrayItems (Total = DefaultCount, all counts = 0). On submit, non-AdHoc tray items are upserted back to the template (match by TrayName) so the defaults stay current.

SiteServiceDocuments

Column	Notes
DocKey	PK
SiteServiceKey	FK

FileName	
MimeType	e.g. 'image/jpeg'
FilePath	path on server file system
TechKey	FK — who uploaded it
FileSize	bytes
UploadedAt	datetime

Save uploaded files to a folder on the server (e.g. `D:\site-services-docs\`), store the path in `FilePath`. The GET `/data` endpoint reads and streams the file back.

Reference tables (existing, no changes)

- `Clients` — ClientKey, ClientName, Address, CityStateZip
- `Departments` — DepartmentKey, ClientKey, DepartmentName
- `Technicians` — TechnicianKey, TechnicianName, Color
- `CalendarEvents` — schedule entries linked to client/dept/tech

API Endpoints

Keep these exact paths and response shapes. The frontend is already built to them.

```
GET    /api/health
GET    /api/technicians
GET    /api/clients
GET    /api/departments?clientId=:id
GET    /api/calendar-events?month=:m&year=:y
POST   /api/calendar-events
PUT    /api/calendar-events/:id
DELETE /api/calendar-events/:id

GET    /api/site-services?technicianId=:id&clientId=:id&status=draft|submitted
GET    /api/site-services/:id
POST   /api/site-services
PATCH  /api/site-services/:id

GET    /api/instrument-types

GET    /api/site-services/:id/trays              — trays with Items[] embedded
POST   /api/site-services/:id/trays              — body: { TrayName, AdHoc,
TemplateKey? }
PATCH  /api/site-services/:id/trays/:trayId      — rename only
DELETE /api/site-services/:id/trays/:trayId      — cascade deletes items

POST   /api/site-services/:id/trays/:trayId/items
PATCH  /api/site-services/:id/trays/:trayId/items/:itemId
DELETE /api/site-services/:id/trays/:trayId/items/:itemId

GET    /api/site-services/:id/documents          — metadata only
GET    /api/site-services/:id/documents/:docId/data  — stream file
POST   /api/site-services/:id/documents          — multipart/form-data upload
DELETE /api/site-services/:id/documents/:docId

GET    /api/tray-templates?clientId=:id&deptId=:id
POST   /api/tray-templates
```

```
PUT    /api/tray-templates/:id
DELETE /api/tray-templates/:id
```

Full request/response shapes are in `server.js` — the mock is the spec.

Key behaviors

- `POST /trays` with `TemplateKey` → copy TrayTemplateItems into SiteServiceTrayItems (Total = DefaultCount, counts = 0)
- `PATCH /site-services/:id` with `Status: 'submitted'` → set DateSubmitted = NOW(), trigger WorkOrderNumber generation if blank
- `GET /trays` response → embed Items[] on each tray object; include CalcedTotal on the service record (SUM of Quantity × UnitPrice)
- All PATCH endpoints are partial — only update fields present in the request body
- Alias `Quantity` → `RepairedCount` on read; reverse on write

Authentication

Steve's part: Add JWT validation middleware to all `/api/*` routes.

1. IT Guy registers the app in Azure AD and gives Steve the **Tenant ID** and **Client ID**
2. Steve adds `passport-azure-ad` (or similar) to validate Bearer tokens on every request
3. The validated token provides the user's UPN (TSI email address)
4. Map UPN to TechKey — either a lookup against the Technicians table by email field, or IT Guy adds TechKey as a custom claim on the token

Dev bypass: Add a check so requests with header `X-Dev-Auth: true` skip validation in local dev. Remove before deploying to production.

What's New vs. Existing

	Existing DB	New
SiteServices, Trays, TrayItems, TrayTemplates	✅ exists	no change
InstrumentTypes, Clients, Departments, Technicians	✅ exists	no change
`AdHoc` column on SiteServiceTrays	❌	add this column
`TrayTemplateItems` table	❌	create this table
`SiteServiceDocuments` table	check — may already exist	create if not

WinScope is unaffected. It reads from the same DB and will simply ignore the two new columns/table it doesn't know about.

IT GUY — Infrastructure

What Needs to Be Set Up

1. Hosting

Host the app on the existing on-prem server (same one running the current portal). The Node/Express app sits behind IIS as a reverse proxy on port 443.

- IIS → reverse proxy → Node app on an internal port (e.g. 3000)
- Or run Node directly on 443 if that's easier in your environment
- Use `pm2` or Windows Service to keep the Node process running

2. Public Access

Techs work on hospital WiFi with no VPN. The app needs to be reachable from the internet.

- Add a DNS record: `field.totalscopeinc.com` → your server's public IP
- Open port 443 (HTTPS) inbound on the firewall
- SSL certificate — Let's Encrypt (free, auto-renews) or use your existing cert provider

The SQL Server does **not** need to be internet-facing. Only the web app layer is exposed.

3. Azure AD App Registration

The app uses Microsoft login (same as Outlook/Teams). You need to register it in Azure AD:

1. Azure Portal → Azure Active Directory → App registrations → New registration
2. Name: `TSI Site Services Field App` (or whatever you want)
3. Redirect URI: `https://field.totalscopeinc.com` (type: Single-page application)
4. Under **API permissions**: add `User.Read` (delegated)
5. Note the **Application (client) ID** and **Directory (tenant) ID** — give these to Steve
6. Optional: create a security group (e.g. "Field Technicians") and restrict the app to that group under Enterprise Applications → Assignment required

4. Technician Email Mapping (coordinate with Steve)

The app identifies techs by their TSI email address from the M365 token. Either:

- Add an email/UPN field to the Technicians table so Steve can do a lookup, **or**
- Add TechKey as a custom claim in the Azure AD app manifest (Steve can advise which is easier for him)

5. Document Storage Folder

Create a folder for uploaded photos/documents (e.g. `D:\site-services-docs\`) and give the app's service account read/write access.

Summary of What Each Person Does

	Steve	IT Guy
Wire SQL Server queries	✅	

JWT auth middleware	✅	
File upload endpoint	✅	
Azure AD app registration		✅
DNS + SSL + firewall		✅
IIS / process manager setup		✅
Docs folder permissions		✅
Technician email mapping	coordinate	coordinate

Questions That Need Answers Before Steve Starts

1. **Exact SQL table names** — the column names in this doc came from the live portal's ASP.NET control IDs. What are the actual table names? (SiteServices? OnsiteService? Something else?)
2. **DepartmentKey = Customer #?** — the live portal's hidden field `hdnDepartmentKey` holds the value shown on screen as "Customer #" (e.g. 8252). Is that correct, or is there a separate ClientKey vs DepartmentKey?
3. **Technician email field** — is there already a UPN/email column on the Technicians table?
4. **WorkOrderNumber generation** — the current portal calls something (stored proc? DB trigger? WinScope API?) to generate the WorkOrderNumber (e.g. NV26006001) when a site service is created. The new API needs to replicate that exact call on `POST /api/site-services`. What is the mechanism — and does it write the number back to the SiteServices record, or return it in a response?
5. **TotalCost vs CalcedTotal** — both exist in the current portal. Is TotalCost a manual override, and do we need it in the new app?
6. **Replaced column** — is it ever populated in real visits, or always 0?
7. **Documents table** — does a `SiteServiceDocuments` table already exist, or is that new?